

LangGraph: From Zero to Production-Ready

Core Mental Model

LangChain = Linear chains ($A \rightarrow B \rightarrow C \rightarrow \text{done}$)

LangGraph = Graphs with loops, branches, state ($A \rightarrow B \rightarrow C \rightarrow A$ again if needed)

Real company AI needs loops, decisions, and multiple agents. LangGraph is how you build that.

Phase 1 — Core Concepts (Sessions 1–3)

Session 1: State + Nodes — shared memory, functions that update state

Session 2: Edges + Conditional Routing — fixed vs conditional edges

Session 3: First LLM Node — LLM as a graph node

Phase 2 — Real Patterns (Sessions 4–6)

Session 4: Loops + Termination — critic node that retries

Session 5: Tool Use — ReAct pattern, web search inside a graph

Session 5.5: MCP — standardized tool access, local MCP server

Session 6: Multi-Agent Coordination — supervisor + specialist agents

Phase 3 — Production Patterns (Sessions 7–9)

Session 7: Persistence + Memory — checkpoints, SQLite/Postgres

Session 8: Human-in-the-Loop — interrupt points, resume from pause

Session 9: Streaming — token streaming + event streaming via SSE

Phase 4 — Production Stack (Sessions 10–13)

Session 10: FastAPI Integration — `/chat` + `/history` endpoints, `app.state`

Session 11: Observability — LangSmith tracing, zero code change

Session 12: MCP with Real Tools — Tavily web search via hosted MCP

Session 13: Docker + Deployment — Dockerize full stack, docker-compose

Final Project — Customer Support Multi-Agent System

Phase 1 — Core System (Complete)

Supervisor agent routes to billing, technical, or general agent. Each specialist has tools via MCP. FastAPI exposes /chat and /history. SSE streaming to client.

Phase 2 — Production Infrastructure (Complete)

SQLite → PostgreSQL for persistent memory across restarts. MCP stdio → streamable-http in separate container. docker-compose orchestrates postgres + mcp-server + fastapi. DNS rebinding protection configured for cross-container communication.

Phase 3 — After Deployment

Human-in-the-loop on billing agent (interrupt_after, /approve endpoint). Auto-retry from checkpoint on agent failure. Per-agent MCP servers (billing tools vs technical tools).

Session: Agent Evaluation (After Deployment)

Build a proper evaluation pipeline for the customer support system.

What we build:

- 25-case dataset for supervisor routing (input → expected next_agent)
- Exact match evaluator for routing accuracy (cheapest, string compare)
- LLM-as-judge evaluator for specialist agent answer quality
- Tool selection eval — did agent call the right tool, or no tool?
- Run with LangSmith evaluate(), get real accuracy score
- Change prompt, rerun, see if score improves

Eval levels: (1) Unit evals per agent, (2) Integration evals full flow, (3) Production evals on real traffic with user feedback.

The cycle: baseline → change → rerun → ship or revert. This is what separates teams that improve their agents from teams that just hope.

Future Work — Advanced Patterns

Auto-Retry from Checkpoint: catch exception → get_state_history() → find last good checkpoint → invoke(None, retry_config). For long-running workflows that should not restart from zero on failure.

Vector Databases: FAISS, Pinecone — semantic search over documents. Gap to fill before BofA start.

Production Stack Architecture

